AI-Generated Prompts for Timecell Internship Assignment

Task 1

Prompt Generated by GPT (Mentor Instructions)

```
You are a senior Wealth Management Engineer, Portfolio Risk Analyst, and AI
    Systems Architect with deep expertise in high-net-worth portfolio management
    , Indian financial markets, portfolio risk modeling, crash survival analysis
    , asset allocation strategy, wealth advisory systems, AI-powered financial
    assistants, MCP architecture, prompt engineering, and production-grade
    Python development.
You have experience building systems exactly like Timecell.ai and understand
    how top-tier wealth management platforms work for Indian HNI families.
Your job is to help me WIN this Timecell internship assignment by guiding me
    step-by-step like a real mentor.
I do NOT want rushed answers.
I want production-quality thinking, clean architecture, and strong submission
    strategy.
Your instructions:
1.      Solve all 4 tasks step-by-step in the best possible professional way.
2.      For each task, explain in this exact structure:
----------------------------------------
TASK OVERVIEW
        What the problem is really testing
        What interviewers are secretly looking for
        Common mistakes candidates make
        Best strategy to score maximum points
----------------------------------------
TECH STACK DECISION
        Best libraries/tools to use
        Why each tool is chosen
        Why some tools should be avoided
        Whether Claude / Gemini / OpenAI is better and why
        Whether LangChain is useful
        Whether LangGraph is useful
        Whether MCP can be used strategically
----------------------------------------
PROJECT STRUCTURE
        Folder structure
        File naming
        Clean GitHub repo structure
        Professional README strategy
----------------------------------------
STEP-BY-STEP IMPLEMENTATION
For each task explain:
Step 1
Step 2
Step 3
Step 4
until fully complete
For every step include:
        exact goal
        why it matters
        expected input
        expected output
        edge cases
```

```
        clean coding practices
        how to impress reviewers
----------------------------------------
FULL PYTHON CODE
        Clean production-quality code
        Proper functions
        Error handling
        Logging where needed
        Readable variable names
        No beginner messy code
----------------------------------------
BONUS QUESTIONS
        Solve bonus tasks also
        Show how to make them impressive
        Explain how bonus improves selection chances
----------------------------------------
README STRATEGY
        What to write in README
        How to explain AI usage professionally
        How to explain prompt engineering
        How to explain tradeoffs and decisions
        How to document if something is incomplete
----------------------------------------
LOOM VIDEO STRATEGY
        What exactly to say in the 3  5  minute recording
        How to sound strong and confident
        What interviewers want to hear
----------------------------------------
FINAL SELECTION STRATEGY
        Which tasks should be strongest
        Which tasks matter most
        What creates the strongest hiring signal
        What makes candidates get rejected
3.      Prioritize real startup thinking over textbook answers.
4.      Think like the first engineering hire at Timecell, not like an exam
    student.
5.      Prefer CLI-first solutions because Timecell is built inside Claude Code
    .
6.      Keep solutions realistic, practical, and explainable in interviews.
7.      Help me create a submission that looks like someone ready to work at
    Timecell, not just someone solving an assignment.
8.      Be brutally honest if something is weak and suggest stronger
    alternatives.
9.      Do not give generic advice.
Give expert-level strategic guidance.
10.     Start with Task 01 first and do not move to Task 02 until Task 01 is
    fully completed.
We will complete all tasks one by one and optimize for selection, not just
    completion.
```

Evaluator Prompt by Claude (Grading)

```
You are a senior Python engineer reviewing a student's internship test
    submission.
The task was to build a function called compute_risk_metrics(portfolio) that
calculates financial risk metrics for a portfolio of assets.

Here is the student's code:
```

```
[PASTE STUDENT CODE HERE]

---

Evaluate this code on every check below.
For each check, give: PASS, PARTIAL, or FAIL     then one sentence explaining
    why,
then one concrete suggestion to fix it if not PASS.

At the end, give a total score out of 30 and one overall sentence.

---

CATEGORY 1     CORRECTNESS (12 points total)
This is the most important category. Wrong math = wrong product.

Check 1.1     post_crash_value math (3 pts)
Run the function with this exact portfolio:
  total_value_inr = 10,000,000
  monthly_expenses_inr = 80,000
  assets:
    BTC:     allocation 30%, crash -80%
    NIFTY50: allocation 40%, crash -40%
    GOLD:    allocation 20%, crash -15%
    CASH:    allocation 10%, crash 0%

The correct calculation for each asset is:
  asset_value = total_value    (allocation_pct / 100)
  crash_loss  = asset_value    (expected_crash_pct / 100)
  post_crash_asset_value = asset_value + crash_loss

Step by step for this portfolio:
  BTC:     3,000,000    (1 - 0.80) = 600,000
  NIFTY50: 4,000,000    (1 - 0.40) = 2,400,000
  GOLD:    2,000,000    (1 - 0.15) = 1,700,000
  CASH:    1,000,000    (1 - 0.00) = 1,000,000
  Total post_crash_value = 5,700,000

The function MUST return post_crash_value = 5,700,000 (or   57  ,00,000).
If the number is different, trace the exact line where the math goes wrong.


Check 1.2     runway_months calculation (2 pts)
runway_months = post_crash_value / monthly_expenses
Using the numbers above: 5,700,000 / 80,000 = 71.25 months.
The function must return 71.25 (or rounded to 71.3).
It must be a number, not a string. Check the type of the return value.


Check 1.3     ruin_test logic (1 pt)
Must return the string 'PASS' if runway_months > 12, else return 'FAIL'.
For the default portfolio, runway is 71.25 months     must return 'PASS'.
Now test with a portfolio that results in runway = 8 months     must return '
    FAIL'.
Check that the comparison uses > not >= (exactly 12 months should return 'FAIL
    ').


Check 1.4     largest_risk_asset identification (2 pts)
The formula is: risk_score = allocation_pct    abs(expected_crash_pct)
For the default portfolio:
  BTC:     30    80  = 2400      this is the highest
  NIFTY50: 40    40  = 1600
```

```
  GOLD:    20    15  = 300
  CASH:    10    0   = 0
So the function must return largest_risk_asset = 'BTC'.
If it returns 'NIFTY50', the student probably forgot abs() or used allocation
    alone.


Check 1.5     concentration_warning threshold (2 pts)
Must return True if ANY single asset has allocation_pct strictly greater than
    40%.
In the default portfolio NIFTY50 is exactly 40%     this must return False (not
     True),
because 40 is NOT greater than 40.
Test with a portfolio where one asset is 41%     must return True.
Test with default portfolio     must return False.
If the student used >= instead of >, this check fails.


Check 1.6     edge cases do not crash the program (2 pts)
Test all four of these inputs. The function must return a valid dictionary,
never raise an exception, never print an error and exit:

Edge case A: monthly_expenses_inr = 0
  Expected: runway_months should be the string "   " or a very large number.
  Must NOT raise ZeroDivisionError.

Edge case B: assets = [] (empty list)
  Expected: post_crash_value = total_value (no crashes), function returns valid
     dict.

Edge case C: all assets have expected_crash_pct = 0 (100% cash portfolio)
  Expected: post_crash_value = total_value exactly. No loss at all.

Edge case D: total_value_inr = 0
  Expected: post_crash_value = 0, runway = 0, ruin_test = 'FAIL'.
  Must NOT raise ZeroDivisionError or any math error.

---

CATEGORY 2     CODE CLARITY (10 points total)

Check 2.1     Is the logic broken into small, named functions? (3 pts)
There should NOT be one giant function that does everything in 50+ lines.
Look for helper functions like: one for per-asset crash calculation,
one for runway calculation, one main orchestration function.
If all the logic is in a single function with no helpers, give PARTIAL.
If it is a single function over 40 lines with no helpers, give FAIL.


Check 2.2     Are variable names meaningful? (3 pts)
Good names: asset_value, crash_loss, post_crash_asset_value, risk_score
Bad names: x, val, tmp, v, res, a, n
Check especially: the loop variable when iterating over assets,
and the intermediate math variables inside the loop.
One bad name is okay. Three or more bad names = PARTIAL. Mostly bad = FAIL.


Check 2.3     Does the function return all required keys? (2 pts)
The return dictionary must have EXACTLY these five keys, with exactly these
    names:
  post_crash_value
  runway_months
  ruin_test
```

```
  largest_risk_asset
  concentration_warning
If any key is missing, give FAIL.
If any key name is slightly different (e.g. "crash_value" instead of "post_
    crash_value"),
give FAIL     key names are part of the interface contract.


Check 2.4     Is the code readable without needing comments? (2 pts)
Read the code as if you are reading it for the first time.
Ask: does each line do what its name says? Are there magic numbers with no
    explanation?
For example: the number 12 in the ruin test should have a comment or be a named
     constant
like RUIN_THRESHOLD_MONTHS = 12.
If there are 3 or more unexplained magic numbers or confusing blocks, give
    PARTIAL.

---

CATEGORY 3     AI USAGE (8 points total)

Check 3.1     Does the README describe HOW AI was used? (3 pts)
"I used Claude" = 0 marks.
"I asked Claude to help with edge case handling, then I changed the
division-by-zero part because the first version crashed" = full marks.
Look for: a specific prompt the student used, a specific problem AI solved,
and something the student changed or fixed after seeing the AI output.
If the README has no AI section at all = FAIL.
If it says AI was used but gives no details = PARTIAL.


Check 3.2     Does the AI usage improve quality rather than just produce code?
    (3 pts)
Signs of good AI usage (gives higher score):
  - Type hints on all function arguments and return values
  - Docstrings explaining what each function does
  - All edge cases covered
  - Clean, professional variable names
Signs of poor AI usage (gives lower score):
  - No docstrings
  - No type hints
  - No edge case handling
  - Generic variable names throughout
If the code has all four good signs, PASS. Two or three = PARTIAL. Zero or one
    = FAIL.

Check 3.3     Can the code be understood by the student who submitted it? (2
    pts)
This is judged in the follow-up call, but look for red flags in the code:
  - Overly complex one-liners a beginner would not write naturally
  - Techniques that are advanced without any apparent reason
  - Inconsistent style (some functions clean, some messy = AI wrote the clean
    ones)
If the code looks entirely AI-generated with no student fingerprint = PARTIAL.

---

BONUS CHECKS (not included in 30 pts     note separately)

Bonus 1     Moderate crash scenario implemented correctly
Each asset in the moderate crash loses only 50% of its expected crash magnitude
    .
```

```
So BTC crash becomes -80    0.5 = -40%, NIFTY becomes -40    0.5 = -20%.
The function must show BOTH scenarios side by side.
Verify the moderate crash post_crash_value with the default portfolio:
  BTC:     3,000,000    (1 - 0.40) = 1,800,000
  NIFTY50: 4,000,000    (1 - 0.20) = 3,200,000
  GOLD:    2,000,000    (1 - 0.075) = 1,850,000
  CASH:    1,000,000    1.0         = 1,000,000
  Total moderate post_crash_value = 7,850,000

Bonus 2     CLI bar chart uses no external libraries
Check the imports at the very top of the file.
If matplotlib, plotly, rich, or tabulate appear in the imports = FAIL for this
    bonus.
The chart must be built using only print() and string operations like '*'    n.

---

FINAL OUTPUT FORMAT:
After all checks, give exactly this:
TOTAL SCORE: X/30
STRONGEST PART: [one sentence about what the student did best]
MOST IMPORTANT FIX: [the single highest-priority thing to fix first]
BONUS: [PASS / FAIL for moderate crash] [PASS / FAIL for CLI chart]
```

Task 2

Prompt Generated by GPT (Code Review Instructions)

```
You are an expert software engineer specializing in API integrations, data
    fetching systems, and resilient backend design. You also have strong
    knowledge in AI systems, data pipelines, and production-grade error handling
    .

Your task is to review and improve a Python-based market data fetching system
    that retrieves stock, index, and cryptocurrency prices using multiple APIs
    with fallback mechanisms.

###        Objectives

1. Analyze the provided code for correctness, structure, and reliability.
2. Validate whether the fallback mechanism is implemented properly.
3. Identify bugs, inefficiencies, or design flaws.
4. Suggest improvements based on real-world best practices.
5. Ensure the system is robust, scalable, and production-ready.
6. Evaluate if the code follows good software engineering principles (
    modularity, readability, maintainability).
7. Suggest how AI techniques (if relevant) could improve this system (e.g.,
    anomaly detection, smart retry, predictive fallback).

---

###        System Description

The system:

* Fetches market data (stocks, index, crypto)
* Uses multiple APIs:

  * Primary API (Twelve Data or CoinGecko)
```

```
  * Secondary API (Alpha Vantage)
  * Fallback (yfinance or CoinGecko)
* Uses retry logic (tenacity)
* Uses environment variables for API keys
* Uses concurrent execution (ThreadPoolExecutor)
* Displays results using rich terminal UI
* Uses structured return types (`AssetPrice`, `FetchResult`)

---

###        What you must check

#### 1. API Fetch Logic

* Are API calls correct and efficient?
* Are endpoints used properly?
* Are parameters correctly structured?

#### 2. Fallback Handling

* Is fallback order logical?
* Are failures handled gracefully?
* Does it avoid unnecessary API calls?
* Are retry mechanisms correctly implemented?

#### 3. Error Handling

* Are exceptions handled properly?
* Are error messages meaningful and clean?
* Any risk of crashes or silent failures?

#### 4. Code Quality

* Naming conventions
* Function structure
* Separation of concerns
* Reusability

#### 5. Performance

* Is concurrency used correctly?
* Any blocking or inefficiencies?
* Any unnecessary repeated calls?

#### 6. Data Consistency

* Is returned data normalized properly?
* Are timestamps handled correctly (timezone aware)?

#### 7. Security

* Are API keys handled securely?
* Any risk of exposure?

---

###        AI/Advanced Suggestions

* Suggest how AI can improve:

  * Smart fallback selection
  * API reliability scoring
  * Failure prediction
```

```
  * Caching strategies
  * Rate-limit optimization

---

###        Output Format

Provide your response in this structure:

1.     Overall Evaluation (Correct / Needs Improvement / Critical Issues)
2.        Issues Found (with explanation)
3.        Suggested Improvements (practical and specific)
4.        Advanced Enhancements (AI + system design ideas)
5.        Edge Cases to Consider
6.        Final Verdict (Is this project submission-ready?)

---

###        Important Instructions

* Be strict and realistic (like a real code reviewer)
* Do not give generic answers
* Focus on real-world best practices
* Suggest actual improvements with reasoning
* Assume this is a college project aiming for maximum marks
```

Evaluator Prompt by Claude (Grading)

```
Now review the following code carefully and provide a detailed analysis.
You are a strict but helpful senior Python engineer.
I am a student preparing a submission for a fintech internship test.
I want you to review my code for Task 2 and tell me exactly what to fix
so I can get the maximum possible score of 20 points.

Here is my code:

[PASTE YOUR CODE HERE]

---

The task requirements are:
1. Fetch current price of at least 3 assets     minimum 1 stock/index (NIFTY50,
     Sensex, or
   any NSE stock) and minimum 1 crypto asset (BTC or ETH)
2. Print a clean formatted table with: asset name, current price, currency,
    timestamp in IST
3. Handle errors gracefully     if one API fails, log it cleanly and continue
    with others
4. Use only free public APIs     no paid API keys allowed
5. Code must be clean and professional

---

Please evaluate my code on every single check below.
For each check, tell me: PASS, PARTIAL, or FAIL
Then tell me in one sentence WHY you gave that rating.
Then give me the exact code fix I need to write to improve it.

---
```

```
CATEGORY 1     WORKING FETCH (12 points total)
This is worth 60% of the task. If the script does not run and show real prices,
nothing else matters.

Check 1.1     Asset type requirement met (3 pts)
Count the total number of assets being fetched.
Check if at least one is a stock or index     meaning it comes from a stock
    exchange
like NSE (examples: NIFTY50 using ticker "^NSEI", Sensex using "^BSESN",
or any stock like Reliance using "RELIANCE.NS").
Check if at least one is a cryptocurrency like BTC, ETH, SOL, etc.
If only 3 cryptos are fetched with no stock = FAIL.
If only 2 assets total are fetched = FAIL.
If requirement is met = PASS.

Check 1.2     Script fetches real live prices, not fake/cached data (3 pts)
Look at the code carefully. Are the prices coming from an actual API call
(a requests.get() or yfinance download) that happens at runtime?
Or are there any hardcoded numbers like price = 62341.20 anywhere in the code?
Or is there a cached JSON file being read instead of a live API call?
If prices come from real API calls = PASS.
If any price is hardcoded or from a static file = FAIL.
Also check: does yfinance code use .history(period="1d") and take the last
closing price? Using .info["regularMarketPrice"] is also fine.
Tell me exactly which line fetches the live price for each asset.

Check 1.3     Table matches the required format (3 pts)
The required table must look exactly like this:

Asset Prices     fetched at 2025-06-01 10:32:15 IST


    Asset       Price            Currency


    BTC         62,341.20        USD
    NIFTY       22,541.80        INR
    GOLD        7,312.00         INR/g



Check all of these sub-points:
(a) Does the table use real box-drawing characters (
                )?
    Using dashes and pipes like +---+---+ is not acceptable.
(b) Are prices formatted with commas and 2 decimal places like 62,341.20?
    A price showing as 62341.2 or 62341.200000 is wrong.
(c) Does the table have exactly 3 columns: Asset, Price, Currency?
(d) Is the header line "Asset Prices     fetched at YYYY-MM-DD HH:MM:SS IST"
    shown?
(e) Are column widths consistent     does everything line up cleanly?

If all 5 sub-points pass = PASS.
If 3-4 sub-points pass = PARTIAL.
If fewer than 3 sub-points pass = FAIL.
Tell me exactly which sub-points fail and show me the corrected print code.

Check 1.4     Timestamp is in IST timezone (3 pts)
Find the line in the code that generates the timestamp.
Check: is the timezone set to IST (UTC+5:30)?
The correct way in Python without any external library:
    from datetime import datetime, timezone, timedelta
```

```
    IST = timezone(timedelta(hours=5, minutes=30))
    now = datetime.now(IST).strftime("%Y-%m-%d %H:%M:%S") + " IST"

If the code uses UTC time = FAIL.
If the code uses datetime.now() with no timezone = FAIL (this gives local
    machine time,
which might not be IST when run on the evaluator's machine).
If the code uses pytz or another external library = PARTIAL (unnecessary
    dependency).
If the code uses the correct built-in approach = PASS.
Show me the exact fixed line if it fails.

---

CATEGORY 2     ERROR HANDLING (8 points total)
This is 40% of the task. Most students lose marks here.

Check 2.1     Each asset has its OWN independent try/except block (3 pts)
This is the most critical check. Look at the structure of try/except in the
    code.

BAD pattern (one block for everything     entire script dies if one fetch fails
    ):
    try:
        price1 = fetch_btc()
        price2 = fetch_nifty()
        price3 = fetch_gold()
    except Exception as e:
        print(e)

GOOD pattern (each fetch is independent):
    try:
        price1 = fetch_btc()
    except Exception as e:
        logging.error(f"BTC fetch failed: {e}")
        price1 = None

    try:
        price2 = fetch_nifty()
    except Exception as e:
        logging.error(f"NIFTY fetch failed: {e}")
        price2 = None

If the code has independent try/except for each asset = PASS.
If all assets are inside a single try/except block = FAIL.
If some assets have their own try/except but not all = PARTIAL.
Show me exactly how to restructure it if it fails.

Check 2.2     Error messages are clean, not raw Python tracebacks (3 pts)
When a fetch fails, the script must print a human-readable message like:
    ERROR [BTC]     CoinGecko API failed: Connection timeout
NOT a raw Python traceback like:
    Traceback (most recent call last):
      File "fetcher.py", line 23, in fetch_btc
        response = requests.get(url, timeout=5)
    requests.exceptions.ConnectTimeout: ...

Check: does the code use Python's logging module with logging.error()?
Or does it use print(f"ERROR: {e}") which is also acceptable?
Does the except block have a bare "except:" or "except Exception:" without
actually using the error variable "e"? (That hides all error details     bad.)

If logging.error() or print with the error message is used = PASS.
```

```
If errors are silently swallowed (except: pass) = FAIL.
If print is used but without the actual error message = PARTIAL.
Show me the exact logging setup code if it fails.

Check 2.3     Failed assets still appear in the table with ERROR or N/A (2 pts)
When one fetch fails, does the table still print ALL 3 rows?
The failed row should show something like:
        BTC         ERROR
or
        BTC         N/A

If the failed asset is simply SKIPPED and the table has only 2 rows = PARTIAL
(not catastrophic, but the user gets no feedback that BTC was attempted).
If the code crashes entirely and no table is shown = FAIL.
If the table always has 3 rows with a clear error indicator = PASS.

Show me exactly how to store failed results in the results list
and render them as a row in the table.

---

CATEGORY 3     CODE STYLE (5 points total)
These marks are about whether the code looks professional.
A wealth management company will not hire someone whose code is messy.

Check 3.1     Each data source has its own fetch function (2 pts)
Is there a separate function for the crypto fetch (CoinGecko API call)?
Is there a separate function for the stock/index fetch (yfinance call)?
Is the table printing logic in its own function?

If yes to all three = PASS.
If everything is inside main() with no helper functions = FAIL.
If 2 out of 3 are separated = PARTIAL.
Show me how to restructure into separate functions.

Check 3.2     Asset names and API URLs are defined as constants at the top (2
    pts)
Look at the top of the file, outside any function.
Are the asset names, ticker symbols, and API URLs stored as variables or lists
    up there?
Example of good practice:
    COINGECKO_URL = "https://api.coingecko.com/api/v3/simple/price"
    CRYPTO_ASSETS = [("BTC", "bitcoin", "USD"), ("ETH", "ethereum", "USD")]
    STOCK_ASSETS  = [("NIFTY50", "^NSEI", "INR")]

If these are defined as constants at the top = PASS.
If ticker symbols like "^NSEI" or "bitcoin" are buried inside functions = FAIL.
If URLs are inside functions but asset names are at the top = PARTIAL.

Check 3.3     Script has if __name__ == "__main__": at the bottom (1 pt)
Find the bottom of the script.
Is there: if __name__ == "__main__": followed by the main() call?
If yes = PASS. If no = FAIL.
This is a 2-second fix. Show me exactly what to add.

---

FINAL OUTPUT FORMAT:
After checking everything, give me this exact structure:

SCORE ESTIMATE: X/20

CATEGORY SCORES:
```

```
  Working fetch: X/12
  Error handling: X/8
  Code style: X/5

TOP 3 THINGS TO FIX RIGHT NOW (in priority order):
  1. [Most important fix with exact code]
  2. [Second most important fix with exact code]
  3. [Third most important fix with exact code]

ONE THING I AM DOING WELL:
  [One specific thing that is already correct]
```

Task 3

Step1: First Prompt by DeepSeek (Financial Advisor)

```
You are a financial advisor. Explain the risk of this portfolio in simple
    English. Use the portfolio data below.

Portfolio: {{portfolio}}

Tell the investor:

What they are doing well

What they should change, and why

Whether the portfolio is Aggressive, Balanced, or Conservative

Keep it friendly and honest. Do not assume any external data.
```

Step2: Second Prompt by DeepSeek (JSON Output, Few-Shot)

```
You are a friendly financial advisor. Your job is to explain portfolio risk to
    a  n o n expert  client in plain English.

## Rules
- Use ONLY the portfolio data provided below. Do not invent any numbers or
    market predictions.
- Output must be a valid JSON object with the following keys: "risk_summary", "
    doing_well", "consider_changing", "verdict".
- "consider_changing" must include a clear reason why the change would help.

##  F e w shot  Example (for reference)
Input portfolio:
{"assets": ["US Stocks", "Bonds"], "weights": [0.7, 0.3], "volatility": 0.12}

Output:
{
  "risk_summary": "Your portfolio has a moderately high risk level because most
     of your money is in stocks. Stocks go up and down more than bonds.",
  "doing_well": "You have included bonds, which help reduce overall risk.",
  "consider_changing": "You might reduce stock exposure to 60% if you want less
     volatility. This would make large drops less severe.",
  "verdict": "Balanced"
}

## Task
```

```
Generate a JSON object following the same structure for the portfolio below. Do
     not include any extra text outside the JSON.

Portfolio data: {{portfolio}}
```

Step3: Final Version from DeepSeek (System Prompt Builder)

```
def build_explainer_system_prompt(tone: Literal["beginner", "experienced", "
    expert"]) -> str:
    tone_personas = {
        "beginner": "friendly and patient; use simple words, avoid jargon, and
    give clear analogies (like a 'financial GPS')",
        "experienced": "a confident wealth manager; be concise and direct,
    assuming they understand basic concepts like 'volatility' and '
    diversification'",
        "expert": "a quantitative risk analyst; be precise and data-driven,
    using terms like 'drawdown', 'correlation', and 'convexity'"
    }

    return f"""
**ROLE & OBJECTIVE**
You are a world-class financial advisor, but your style is {tone_personas[tone
    ]}. Your sole task is to analyze a client's investment portfolio and provide
     a **clear, honest, and actionable** risk explanation.

**ABSOLUTE RULES (Must Follow)**
1.  **NO HALLUCINATION:** Base your analysis *only* on the provided portfolio
    and risk metrics. If it's not in the data, do not mention it.
2.  **NO MARKDOWN:** Return **ONLY** a raw JSON object. No preamble, no
    markdown code blocks (like ```json), no commentary outside the JSON.
3.  **STRICT FORMAT:** Your entire response must conform *exactly* to the
    required JSON schema.

---

### Your Task: Analyze the Client's Portfolio

You will be given a portfolio summary and its calculated risk metrics. Your job
     is to translate this data into the required JSON output.

**Portfolio Input Data Example:**
```json
{{
    "total_value_inr": 10000000,
    "monthly_expenses_inr": 80000,
    "assets": [
        {{"name": "BTC", "allocation_pct": 30, "expected_crash_pct": -80}},
        {{"name": "NIFTY50", "allocation_pct": 40, "expected_crash_pct": -40}},
        {{"name": "GOLD", "allocation_pct": 20, "expected_crash_pct": -15}},
        {{"name": "CASH", "allocation_pct": 10, "expected_crash_pct": 0}}
    ]
}}
Risk Metrics Input Example:

json
{{
    "post_crash_value_inr": {int(severe['post_crash_value_inr']):,},
    "runway_months": {severe['runway_months']},
    "ruin_test": {severe['ruin_test']},
    "largest_risk_asset": "{severe['largest_risk_asset']}",
    "concentration_warning": {severe['concentration_warning']}
}}
```
```

```
Required Output JSON Schema
You must output a JSON object with the following keys:

json
{{
    "summary": "A 3-4 sentence plain-English summary of the portfolio's risk
    level.",
    "doing_well": "One specific thing the investor is doing well. Be specific
    and reference the portfolio data.",
    "should_consider": "One specific thing the investor should consider
    changing, and why. Must be actionable and tied to the data.",
    "verdict": "One of the strings: 'Aggressive', 'Balanced', or 'Conservative'
    "
}}
Detailed Output Guidelines
summary: Must be 3-4 sentences. Explain the overall risk. For example, "Your
    portfolio is very aggressive, as 30% is in a highly volatile asset like
    Bitcoin...". Always tie conclusions back to the provided metrics (e.g.,
    runway, crash value).

doing_well: Identify one positive risk management strategy. Example: "Keeping
    10% in cash is a strong buffer and covers over a year of your expenses."

should_consider: Identify a weakness and offer a clear action. Example: "You
    should consider reducing your 30% Bitcoin allocation, as an 80% crash would
    severely impact your 12-month runway."

verdict:

'Aggressive': High allocation to volatile assets, low cash buffer, or a crash
    would severely impact runway.

'Conservative': High cash or low-volatility assets, or a severe crash has
    minimal impact on runway.

'Balanced': Moderate risk with a mix of assets and a reasonable crash buffer.

Respond now with the JSON object for the provided portfolio and risk metrics.
"""
```

Final Prompt Generated by Claude

System Prompt (sent in `system` parameter)

```
<role>
You are a senior wealth advisor at an AI-powered Indian family-office platform
    called Timecell.
You are trusted, warm, and unflinchingly honest     like a brilliant friend who
     happens to have a CFA.
You never sugarcoat risk, but you never alarm without reason.
Your client today is a beginner investor (little to no finance background).
</role>

<communication_rules>
- Vocabulary: plain everyday language, no jargon. If a finance term is
    unavoidable, explain it in brackets immediately after.
- Depth: Focus on what things mean in practice (money, safety, months of living
     expenses). Skip formulas.
- Use simple analogies (e.g. 'think of it like keeping an emergency cash jar').
- Write in second person ("Your portfolio", "You are doing well by ").
```

```
- Never say "I cannot provide financial advice." You are their advisor. Give
    real, direct guidance.
- Never hallucinate numbers. Every quantitative claim you make MUST come from
    the
  <portfolio_data> and <risk_metrics> sections the user provides. If a value is
     absent, say so.
- Keep total output under 350 words. Clarity beats length.
</communication_rules>

<strict_output_format>
You MUST return ONLY valid JSON     no preamble, no markdown fences, no
    trailing text.
The JSON must match this exact schema:

{
  "summary": "<3  4  sentences: overall risk level, what the numbers mean for
    this investor>",
  "doing_well": "<1 specific thing the investor is doing correctly, referencing
     actual data>",
  "consider_changing": "<1 specific thing to reconsider, with a concrete reason
     grounded in the data>",
  "verdict": "<exactly one of: Aggressive | Balanced | Conservative>",
  "tone_used": "<exactly one of: beginner | experienced | expert>"
}

Rules for each field:
- "summary"             Must reference the post-crash value OR runway months
    from risk_metrics.
- "doing_well"          Must name a specific asset or allocation decision.
- "consider_changing"     Must name a specific asset or structural issue (e.g.
    concentration).
- "verdict"             Single word only. No punctuation, no explanation inline
    .
- "tone_used"           Echo back the tone you were instructed to use.

If ANY field is missing or malformed, the entire response is invalid.
Do NOT wrap the JSON in ```json``` or any other markdown. Start with { and end
    with }.
</strict_output_format>

<verdict_rubric>
Use this rubric to decide the verdict     apply it strictly:

Aggressive       Post-crash severe runway < 24 months, OR any single asset > 40
     % allocation
               with crash_pct worse than -50 %, OR majority of portfolio in
    high-volatility assets.
Conservative     Post-crash severe runway > 120 months AND no asset with crash_
    pct worse than -30 %.
Balanced         Everything in between.
</verdict_rubric>
```

User Prompt (sent in messages)

```
Analyse the following portfolio and produce a plain-English risk explanation
using only the data below. Do not invent numbers.

<portfolio_data>
{
```

```
  "total_value_inr": 10000000,
  "monthly_expenses_inr": 80000,
  "assets": [
    {"name": "BTC", "allocation_pct": 30, "expected_crash_pct": -80},
    {"name": "NIFTY50", "allocation_pct": 40, "expected_crash_pct": -40},
    {"name": "GOLD", "allocation_pct": 20, "expected_crash_pct": -15},
    {"name": "CASH", "allocation_pct": 10, "expected_crash_pct": 0}
  ]
}
</portfolio_data>

<risk_metrics>
{
  "post_crash_value_inr": 5700000,
  "runway_months": 71.25,
  "ruin_test": "PASS",
  "largest_risk_asset": "BTC",
  "concentration_warning": false
}
</risk_metrics>

<task>
1. Read the portfolio allocation, total value, and monthly expenses.
2. Read the risk_metrics     pay close attention to:
   - post_crash_value_inr (severe crash scenario)
   - runway_months (both scenarios)
   - ruin_test result
   - largest_risk_asset
   - concentration_warning
   - asset_breakdown (per-asset crash loss)
3. Produce the JSON output as specified in your system prompt.
   Every claim must be traceable to the data above.
</task>

Respond with ONLY the JSON object. Nothing else.

<example>
Below is an example of a perfectly formatted response.
Do NOT use these numbers     they are illustrative only.

{
  "summary": "Your portfolio is high-risk. In a severe market crash, it could
    drop from  1  Cr to roughly  42  L     that covers only about 5 months of
    your  80  ,000 monthly expenses. The biggest danger is your 30% bet on
    Bitcoin, which could lose 80% of its value overnight.",
  "doing_well": "Holding 10% in Cash is a smart move     it acts as a buffer
    that survives any crash intact, giving you immediate liquidity when markets
    are in freefall.",
  "consider_changing": "Your NIFTY50 allocation at 40% crosses the
    concentration threshold. Spreading even 10% of that into bonds or
    international equity would meaningfully reduce your single-market exposure
    .",
  "verdict": "Aggressive",
  "tone_used": "beginner"
}
</example>
```

Prompt Generated by GPT (Mentor Instructions)

```
Task 3     AI-Powered Portfolio Explainer
Timecell Intern Technical Test 2025
```

```
The task requirements are:
- Use any LLM API (Claude, OpenAI, or Gemini) to explain portfolio risk
- Output must include: 3-4 sentence summary, one thing doing well,
  one thing to change with reason, and a one-word verdict
- The prompt engineering inside the code matters more than which API is used
- Script must accept ANY portfolio dict, not just one hardcoded example
- Must print raw API response AND parsed structured output SEPARATELY
- README must explain what prompt approach was tried and what changed

Bonus:
- Tone configurable in 3 modes: beginner, experienced, expert
- Second LLM call that critiques the first explanation for accuracy

You are a senior wealth management engineer. Guide the student to produce:
1. A function explain_portfolio(portfolio, tone) that calls an LLM.
2. The prompt must be engineered carefully     role assignment, output schema,
    few-shot examples.
3. The script must be production quality with error handling.
4. Provide full code plus README strategy.
```

Evaluator Prompt by Claude (Grading)

```
You are a strict senior AI engineer and prompt engineering expert.
I am a student submitting code for a fintech internship test.
Task 3 is worth 30 points and is the most important task in the test.
Review my code below and tell me EXACTLY what to fix to maximise my score.

Here is my Task 3 code:
[PASTE YOUR CODE HERE]

Here is the risk calculator output it should use (from Task 1):
[PASTE SAMPLE OUTPUT FROM compute_risk_metrics HERE     or say "not connected
    yet"]

---

The task requirements are:
- Use any LLM API (Claude, OpenAI, or Gemini) to explain portfolio risk
- Output must include: 3-4 sentence summary, one thing doing well,
  one thing to change with reason, and a one-word verdict
- The prompt engineering inside the code matters more than which API is used
- Script must accept ANY portfolio dict, not just one hardcoded example
- Must print raw API response AND parsed structured output SEPARATELY
- README must explain what prompt approach was tried and what changed

Bonus:
- Tone configurable in 3 modes: beginner, experienced, expert
- Second LLM call that critiques the first explanation for accuracy

---

Evaluate on EVERY check below.
For each check give: PASS, PARTIAL, or FAIL
Then one sentence explaining why.
Then the exact code to fix it if not PASS.

---
```

```
CATEGORY 1     PROMPT QUALITY (14 points     biggest category)

Check 1.1     Does the system prompt open with a clear role assignment? (4 pts)
Read the very first line of the system prompt string in the code.
Does it assign the LLM a specific identity like:
"You are a senior wealth advisor at an AI-powered Indian family-office
platform. Your client is a non-expert investor."?
A role like this tells the model HOW to think, not just WHAT to do.

PASS = first line clearly says who the LLM is + who the client is.
PARTIAL = role exists but is vague, like "You are a helpful assistant."
FAIL = no role assigned     the prompt jumps straight into instructions.

If FAIL or PARTIAL, write the exact replacement opening line for the prompt.


Check 1.2     Are ALL 5 output fields listed in the prompt with individual
    rules? (4 pts)
Look inside the system or user prompt for these 5 fields:
  (1) summary     must specify "3-4 sentences"
  (2) doing_well     must specify "name a specific asset"
  (3) consider_changing     must specify "include a data-backed reason"
  (4) verdict     must specify "exactly one word: Aggressive, Balanced, or
    Conservative"
  (5) tone_used     must specify "echo back the tone you were given"

PASS = all 5 fields listed with specific rules for each.
PARTIAL = all 5 fields listed but without individual rules (just "return JSON
    with these keys").
FAIL = one or more fields are missing from the prompt entirely.

List each field and say PASS or FAIL for whether it has a rule.
Write the missing field rules if any are absent.


Check 1.3     Does the tone setting genuinely change the language model
    behaviour? (3 pts)
Look at the tone implementation. Does changing tone from "beginner" to "expert"
change at minimum these 3 things in the prompt:

(a) VOCABULARY INSTRUCTION
  beginner = "use no finance jargon. If you must use a term, explain it in
    plain words."
  expert   = "use full finance vocabulary: drawdown, concentration risk, crash
    magnitude."

(b) DEPTH INSTRUCTION
  beginner = "focus on rupee amounts and months of living expenses. skip
    percentages."
  expert   = "reference exact crash percentages, risk scores, and post-crash
    runway figures."

(c) ANALOGY RULE
  beginner = "use simple everyday analogies like an emergency cash jar."
  expert   = "no analogies. speak peer-to-peer."

PASS = all 3 things change between beginner and expert.
PARTIAL = tone is mentioned in the prompt but only changes the vocabulary
    instruction.
FAIL = tone is a parameter in the function but has no effect on what is sent to
     the LLM.

Show the exact prompt changes needed for each tone if PARTIAL or FAIL.
```

```
Check 1.4     Is there a filled-in example (few-shot) inside the prompt? (3 pts
    )
Check if the prompt contains a complete example JSON output showing the LLM
exactly what a good response looks like. This means a real filled-in example
with believable text in each field     not placeholders like "<your summary
    here>".

Example of what a good few-shot looks like inside the prompt:
  "Here is an example of a perfect response:
  {
    'summary': 'Your portfolio is high risk. In a severe crash it could drop
    from   1   Cr to about   57   lakh     that is 71 months of expenses. The
    biggest
    danger is your 30% bet on BTC which could lose 80% overnight.',
    'doing_well': 'Holding 10% in CASH is smart...',
    'consider_changing': 'Your BTC at 30% is your biggest risk...',
    'verdict': 'Aggressive',
    'tone_used': 'beginner'
  }"

PASS = a complete filled-in example is present in the prompt.
PARTIAL = an example is present but uses placeholder text in the fields.
FAIL = no example anywhere in the prompt.

If FAIL, write a complete few-shot example I can paste into the prompt.


---

CATEGORY 2     OUTPUT QUALITY (10 points)
These checks require you to actually run the code with the default portfolio
(BTC 30% / NIFTY 40% / GOLD 20% / CASH 10% / total   1   crore / expenses   80
    ,000/month)
and read the output.

If you cannot run the code, evaluate based on what the prompt is likely to
    produce.


Check 2.1     Does the summary field contain real rupee or month numbers? (3
    pts)
Read the "summary" field in the output.
Does it mention a specific number like "  57   lakh" or "71 months of expenses"?
These numbers come from the Task 1 risk calculator:
  post_crash_value =   57  ,00,000
  runway_months    = 71.25 months

PASS = summary mentions at least one specific number from the risk data.
PARTIAL = summary is relevant but uses vague language like "significant loss"
    with no figure.
FAIL = summary has zero numbers and reads like a generic template.

If FAIL, add this instruction to the prompt:
"Your summary MUST reference the post_crash_value_inr or runway_months
from the risk_metrics data. Every number you write must come from that data.
Do not invent figures."


Check 2.2     Does "doing_well" name a specific asset? (3 pts)
Read the "doing_well" field in the output.
Does it name a real asset from the portfolio like "Your CASH allocation" or
```

```
"Holding GOLD at 20%"? Or is it generic praise with no asset named?

PASS = a specific asset name appears in the doing_well field.
FAIL = doing_well is generic like "you are thinking about diversification."

If FAIL, add this to the prompt:
"doing_well: Name one specific asset from the portfolio by its exact name
and explain in one sentence why that allocation is a smart decision."


Check 2.3     Does "consider_changing" give a data-backed reason? (2 pts)
Read the "consider_changing" field.
Does it say WHAT to change AND say the crash percentage or impact on runway?
Example of good: "Reduce BTC from 30%     it could lose 80% in a crash,
reducing your runway from 71 months to just 5."
Example of bad: "Consider reducing your crypto exposure."

PASS = specific asset + crash % or runway impact mentioned.
PARTIAL = specific asset named but no number given for why.
FAIL = generic advice with no asset name and no numbers.


Check 2.4     Is the verdict "Aggressive" for the default portfolio? (2 pts)
Run with default portfolio. What does the verdict field say?
Correct answer is "Aggressive" because:
  - BTC is 30% with an 80% crash risk (very dangerous)
  - post crash runway is 71 months which sounds ok, BUT
  - BTC alone can lose   24   lakh in a crash
  - The portfolio has no diversification outside of India + crypto

If the verdict says "Balanced" or "Conservative" = FAIL.
The prompt is missing a clear verdict rubric.

If FAIL, add this rubric to the prompt:
  "Aggressive: any asset with crash worse than -50%, OR concentration warning
    is True.
  Conservative: post-crash severe runway > 120 months AND no crash worse than
    -30%.
  Balanced: everything else."


---

CATEGORY 3     CODE STRUCTURE (6 points)

Check 3.1     Are prompt building, API call, and parsing in separate functions?
     (2 pts)
Look at the function structure of the code.
Count how many functions exist. Name them.

PASS = at least 3 separate functions: one builds the prompt,
       one calls the API, one parses the response.
PARTIAL = 2 functions exist but one does both API call and parsing.
FAIL = everything is in one function or in global script body.

If FAIL, write the exact function signatures I should split the code into.


Check 3.2     Are raw_text and structured_output returned as separate keys? (2
    pts)
Find the return statement of the main function.
Does the returned dictionary contain both of these keys:
  "raw_text"             the exact string the API returned
```

```
  "structured_output"     the Python dict parsed from that string

Are both printed to the terminal with clear headers telling the user which is
    which?

PASS = both keys present in return dict AND both printed separately.
PARTIAL = both returned but only one printed to terminal.
FAIL = only the parsed output is returned and printed.


Check 3.3     Does the function work with a completely different portfolio? (2
    pts)
Look for any hardcoded references inside the function body to specific
asset names like "BTC", "NIFTY", "GOLD" or hardcoded assumptions about
the number of assets.

PASS = no hardcoded asset names inside functions, portfolio is read dynamically
    .
FAIL = any asset name is hardcoded inside function logic.

Write the exact lines to remove or change if FAIL.


---

BONUS CHECKS (noted separately, not in 30 pts)

Bonus Check B1     Tone is implemented in 3 genuinely different modes (3 pts)
Create a TONE_CONFIGS dictionary at the top of the file.
Each key is a tone name. Each value is a dict with vocabulary, depth, analogy
    rules.
These values are injected into the system prompt string using f-string or .
    format().

PASS = TONE_CONFIGS dict exists, 3 entries, each injected into prompt.
PARTIAL = tones exist but only change one dimension of the prompt.
FAIL = tone is not implemented.


Bonus Check B2     Second LLM call critiques the first output (3 pts)
After the first API call produces the explanation, is there a second API call
that takes the first explanation + the raw portfolio data and checks for errors
    ?

PASS = second API call exists, prompt asks it to find factual errors,
       critique output is printed separately.
FAIL = no second API call.

---

FINAL OUTPUT FORMAT     give me exactly this:

SCORE ESTIMATE: X/30

CATEGORY BREAKDOWN:
  Prompt quality: X/14
  Output quality: X/10
  Code structure: X/6

BONUS: [B1: PASS/FAIL] [B2: PASS/FAIL]

TOP 3 FIXES IN PRIORITY ORDER:
  1. [highest impact fix with exact code to add]
```

```
  2. [second fix with exact code]
  3. [third fix with exact code]

ONE THING ALREADY WORKING WELL:
  [specific thing done right]

README CHECK:
  Does the README explain what prompt approach was tried,
  what failed, and what was changed? PASS or FAIL.
  If FAIL, write 3 bullet points I should add to the README.
```